# RAPPORT DE TÂCHE

## Mission 5 — Tâche 1 : Corriger des problèmes de sécurité

*Projet MediatekDocuments — C# / PHP*

| Projet | MediatekDocuments — BTS SIO SLAM |
|---|---|
| **Mission** | Mission 5 — Sécurité et qualité |
| **Tâche** | Tâche 1 — Corriger des problèmes de sécurité |
| **Temps estimé** | 2 heures |
| **Temps réel** | 2 heures |
| **Dépôt C#** | orient75015/mediatekdocuments — branche fix/securite-credentials-appsettings |
| **Dépôt PHP** | orient75015/rest_mediatekdocuments — branche fix/securite-acces-direct-api |
| **Fichiers modifiés** | App.config, Access.cs (dépôt C#), .htaccess (dépôt PHP) |

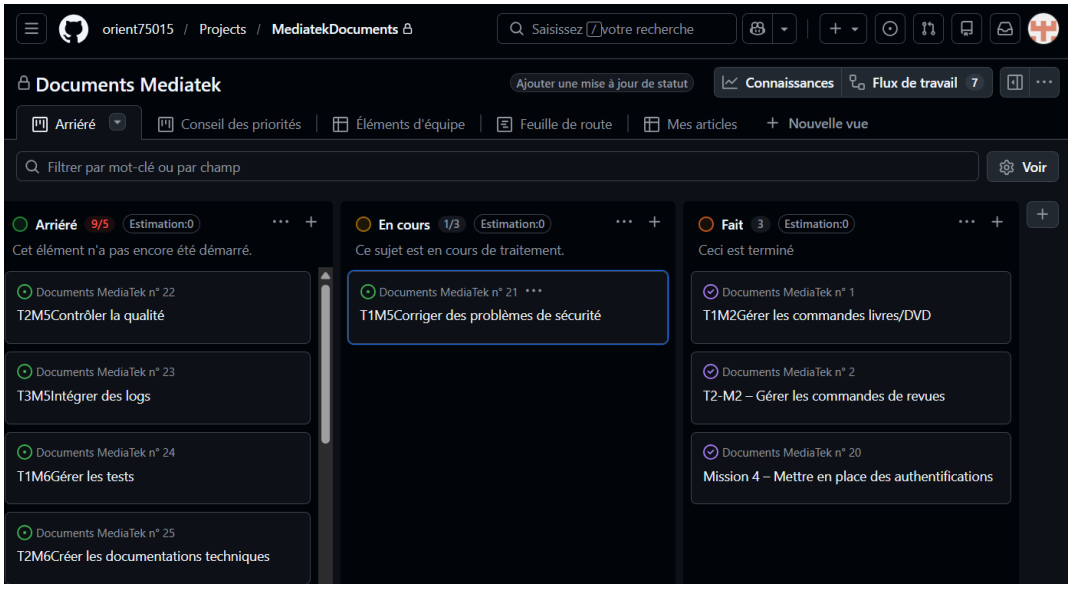
## 1. Demande de la tâche

Deux problèmes de sécurité ont été identifiés dans l'application MediatekDocuments :

- Problème 1 : le couple login:pwd de l'API est codé en dur dans Access.cs, ce qui représente un risque de sécurité si le code source est partagé.
- Problème 2 : une requête GET vide sur l'API ne retourne pas d'erreur, laissant l'accès direct à l'API sans paramètre possible.

Pour chaque problème, une issue GitHub a été créée, une branche de correction a été développée, puis un pull request a été soumis et accepté.

## 2. Kanban




*Capture du Kanban GitHub — T1M5 dans la colonne 'In Progress'*

## 3. Temps estimé et temps réel

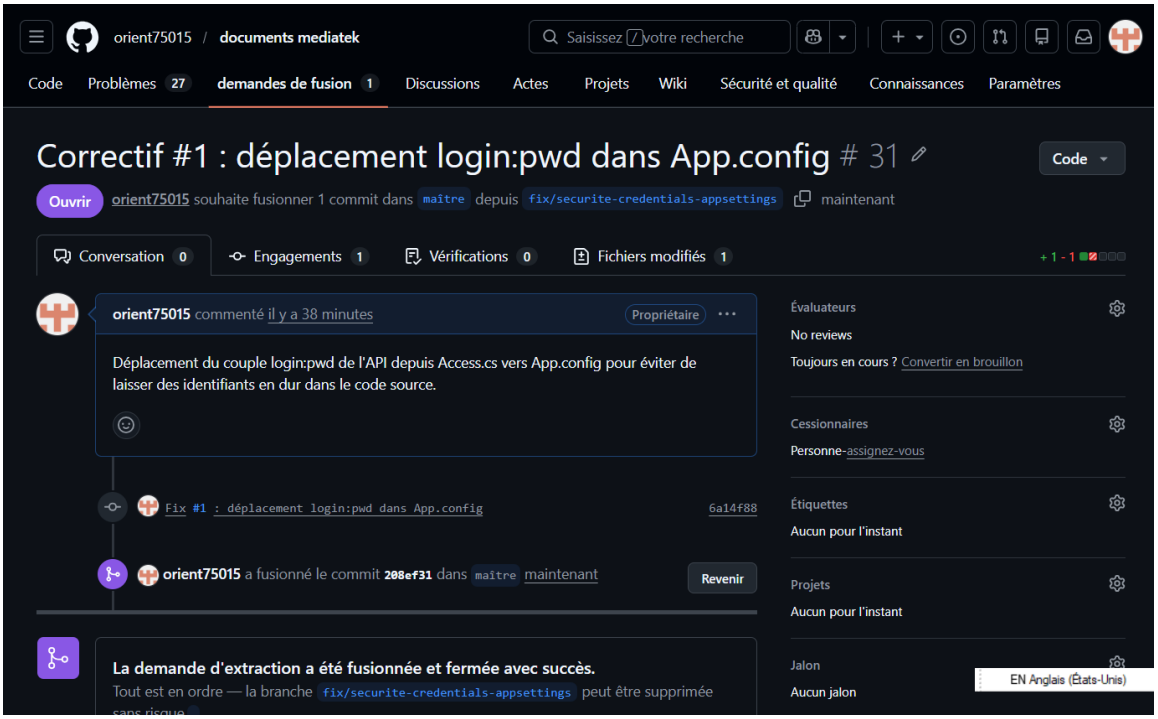
| Indicateur | Valeur |
|---|---|
| **Temps estimé** | 2 heures |
| **Temps réel** | 2 heures |
| **Écart** | Aucun |

## 4. Problème 1 — Login:pwd en dur dans Access.cs

### 4.1 Issue GitHub

*Capture de l'issue GitHub #1 — 'Sécurité : login:pwd en dur dans Access.cs' (dépôt mediatekdocuments)*

## 4.2 Pull request



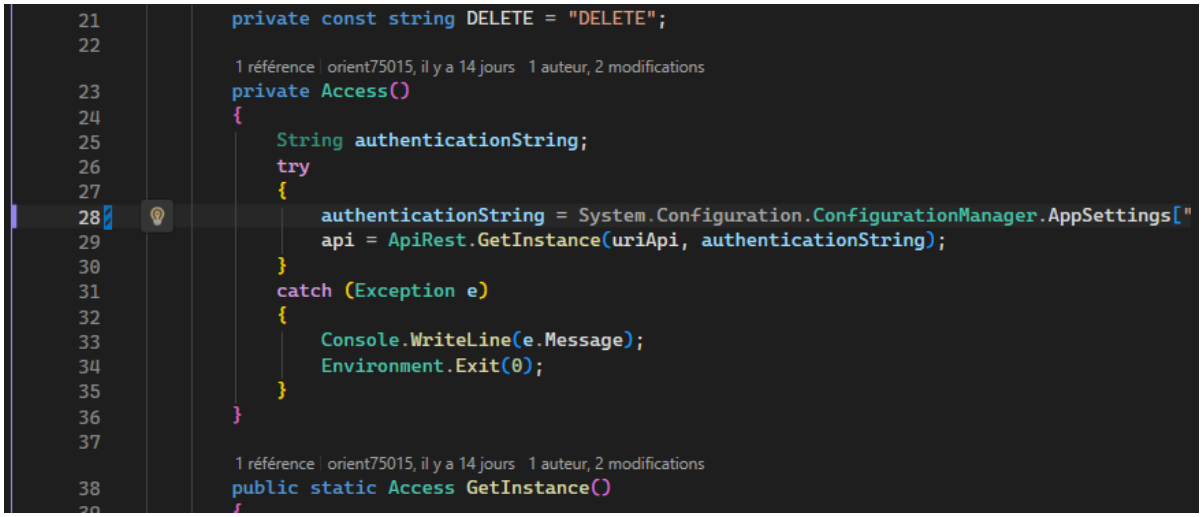

*Capture du pull request mergé — branche fix/securite-credentials-appsettings vers master*

## 4.3 Modification du code

Avant la correction, le couple login:pwd était codé en dur directement dans le fichier Access.cs :

Fichier Access.cs — AVANT :

```
authenticationString = "admin:adminpwd";
```



```
21      private const string DELETE = "DELETE";
22
        1 référence | orient75015, il y a 14 jours | 1 auteur, 2 modifications
23      private Access()
24      {
25          String authenticationString;
26          try
27          {
28              authenticationString = System.Configuration.ConfigurationManager.AppSettings["
29              api = ApiRest.GetInstance(uriApi, authenticationString);
30          }
31          catch (Exception e)
32          {
33              Console.WriteLine(e.Message);
34              Environment.Exit(0);
35          }
36      }
37
        1 référence | orient75015, il y a 14 jours | 1 auteur, 2 modifications
38      public static Access GetInstance()
```

*Capture de Access.cs — ligne authenticationString avant correction (dal\Access.cs)*

La correction consiste à déplacer cette valeur dans le fichier App.config, qui est un fichier de configuration externe non versionné ou protégé, en s'inspirant de l'application Habilitations.

Fichier App.config — APRÈS (ajout dans <appSettings>) :

```
<appSettings>
  <add key="apiCredentials" value="admin:adminpwd"/>
</appSettings>
```

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
    <configSections>
    </configSections>
    <startup>
        <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.7.2" />
    </startup>
  <runtime>
    <assemblyBinding xmlns="urn:schemas-microsoft-com:asm.v1">
      <dependentAssembly>
        <assemblyIdentity name="Newtonsoft.Json" publicKeyToken="30ad4fe6b2a6aeed" culture=
        <bindingRedirect oldVersion="0.0.0.0-13.0.0.0" newVersion="13.0.0.0" />
      </dependentAssembly>
    </assemblyBinding>
  </runtime>
</configuration>
<appSettings>
    <add key="apiCredentials" value="admin:adminpwd"/>
  </appSettings>
```

*Capture de App.config avec la clé apiCredentials ajoutée*

Fichier Access.cs — APRÈS :

```
authenticationString = System.Configuration.ConfigurationManager
                           .AppSettings["apiCredentials"];
```

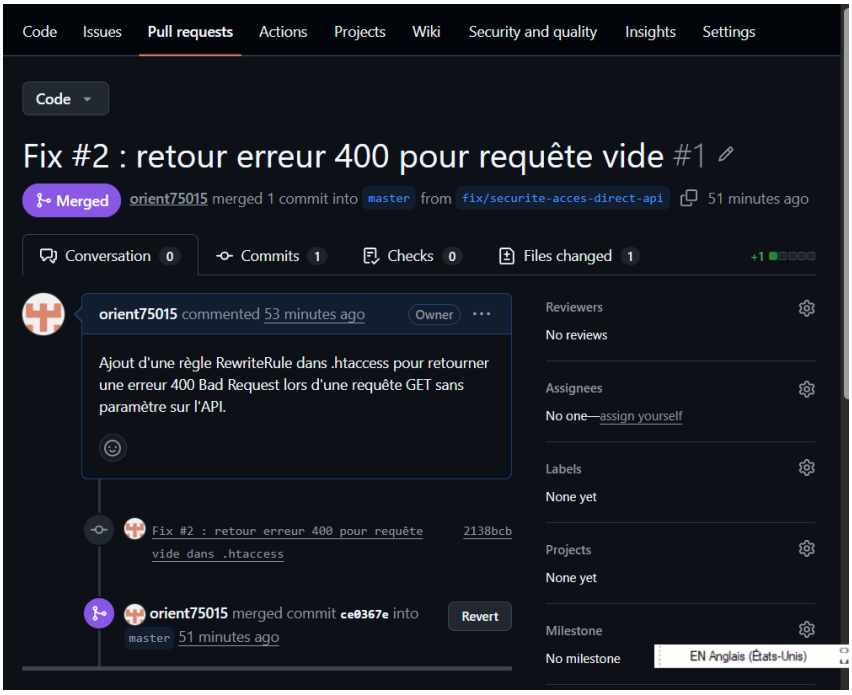
```
28              authenticationString = System.Configuration.ConfigurationManager.AppSettings["apiCredentials"];
29              api = ApiRest.GetInstance(uriApi, authenticationString);
30          }
31          catch (Exception e)
32          {
33              Console.WriteLine(e.Message);
34              Environment.Exit(0);
35          }
36      }
37
        1 référence | orient75015, il y a 14 jours | 1 auteur, 2 modifications
38      public static Access GetInstance()
39      {
40          if (instance == null)
41          {
42              instance = new Access();
43          }
44          return instance;
45      }
46
        1 référence | orient75015, il y a 14 jours | 1 auteur, 2 modifications
47      public List<Categorie> GetAllGenres()
48      {
49          IEnumerable<Genre> lesGenres = TraitementRecup<Genre>(GET, "genre", null);
50          return new List<Categorie>(lesGenres);
```

*Capture de Access.cs — ligne authenticationString après correction (dal\Access.cs)*

Explication : en déplaçant les identifiants dans App.config, le code source ne contient plus de données sensibles en clair. Le fichier App.config peut être exclu du versioning via .gitignore ou protégé séparément.

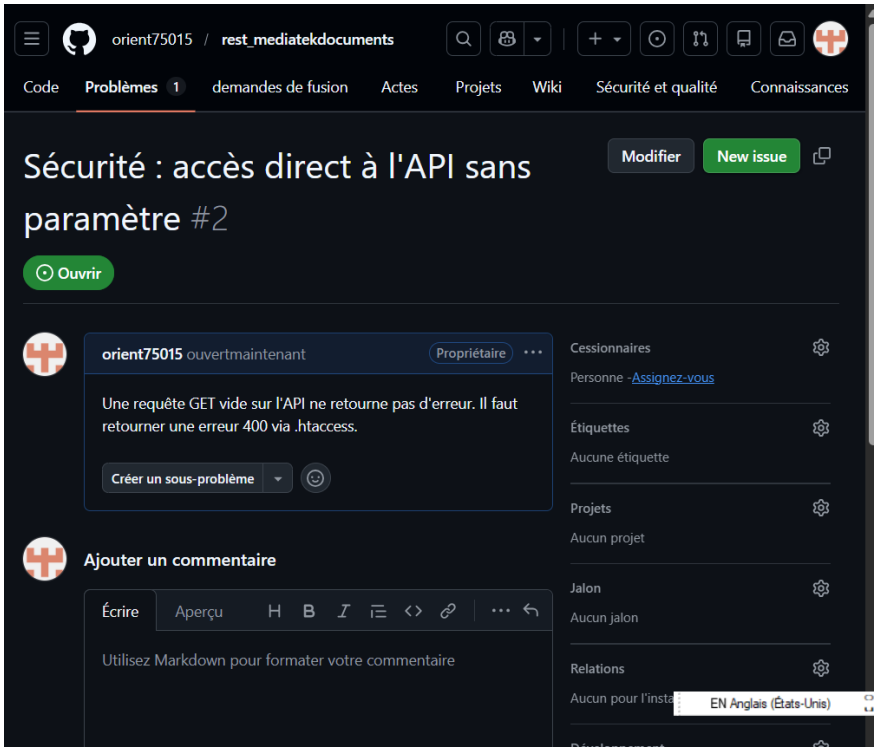
## 5. Problème 2 — Accès direct à l'API sans paramètre

### 5.1 Issue GitHub




*Capture de l'issue GitHub #1 — 'Sécurité : accès direct à l'API sans paramètre' (dépôt rest_mediatekdocuments)*

### 5.2 Pull request




*Capture du pull request mergé — branche fix/securite-acces-direct-api vers master*

### 5.3 Modification du code

Avant la correction, une requête GET vide sur l'URL de l'API (sans paramètre table) retournait la liste des fichiers du dossier ou une erreur 403 selon la configuration Apache, mais pas une erreur 400 explicite.

Fichier .htaccess — AVANT :

```
Options -Indexes
RewriteEngine on
RewriteCond %{REQUEST_METHOD} =GET
RewriteRule ^([a-zA-Z0-9_]+)$ src/index.php?table=$1 [B,L]
...
```

Fichier .htaccess — APRÈS (ajout de la règle 400) :

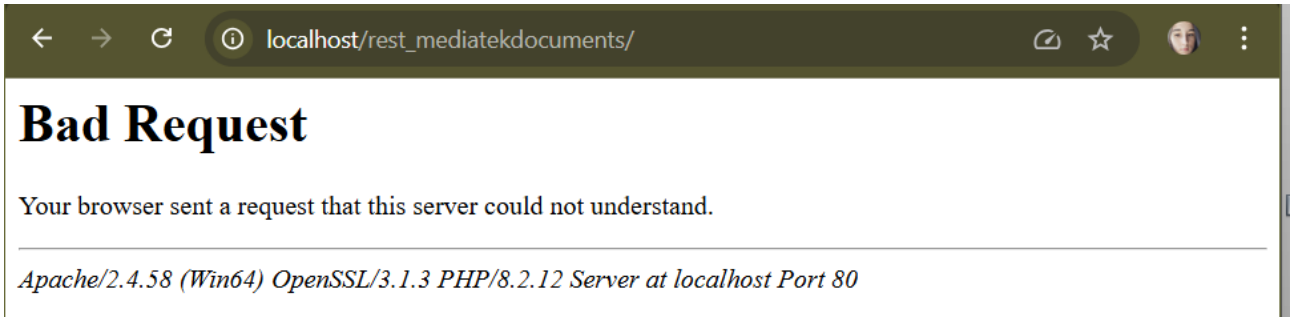
```
Options -Indexes
RewriteEngine on
RewriteRule ^$ - [R=400,L]
RewriteCond %{REQUEST_METHOD} =GET
RewriteRule ^([a-zA-Z0-9_]+)$ src/index.php?table=$1 [B,L]
...
```

```
1   Options -Indexes
2   RewriteEngine on
3   RewriteRule ^$ - [R=400,L]
4   RewriteCond %{REQUEST_METHOD} =GET
5   RewriteRule ^([a-zA-Z0-9_]+)/([a-zA-Z0-9]+)$ src/index.php?table=$1/$2 [B,L]
6   RewriteCond %{REQUEST_METHOD} =GET
7   RewriteRule ^([a-zA-Z0-9_]+)$ src/index.php?table=$1 [B,L]
8   RewriteCond %{REQUEST_METHOD} =GET
9   RewriteRule ^([a-zA-Z0-9_]+)/({.*})$ src/index.php?table=$1&champs=$2 [B,L]
10  RewriteCond %{REQUEST_METHOD} =POST
11  RewriteRule ^([a-zA-Z0-9_]+)$ src/index.php?table=$1 [B,L]
12  RewriteCond %{REQUEST_METHOD} =PUT
13  RewriteRule ^([a-zA-Z0-9_]+)$ src/index.php?table=$1 [B,L]
14  RewriteCond %{REQUEST_METHOD} =PUT
15  RewriteRule ^([a-zA-Z0-9_]+)/([a-zA-Z0-9]+)$ src/index.php?table=$1&id=$2 [B,L]
16  RewriteCond %{REQUEST_METHOD} =DELETE
17  RewriteRule ^([a-zA-Z0-9_]+)$ src/index.php?table=$1 [B,L]
18  RewriteCond %{REQUEST_METHOD} =DELETE
19  RewriteRule ^([a-zA-Z0-9_]+)/({.*})$ src/index.php?table=$1&champs=$2 [B,L]
```

*Capture du fichier .htaccess dans NetBeans avec la ligne RewriteRule ^$ - [R=400,L] visible*

Explication : la règle RewriteRule ^$ - [R=400,L] intercepte toute requête dont l'URL est vide (après le nom du dossier de l'API) et retourne immédiatement une erreur HTTP 400 Bad Request. Cela empêche tout accès direct à l'API sans paramètre table.

Résultat après correction — accès à http://localhost/rest_mediatekdocuments/ :




*Capture du navigateur affichant '400 Bad Request' après la correction*

## 6. Résultat

Les 2 problèmes de sécurité sont corrigés et les solutions ont été envoyées en pull request et validées sur GitHub :

- Problème 1 : le couple login:pwd est maintenant stocké dans App.config et récupéré via ConfigurationManager.AppSettings. Le code source ne contient plus d'identifiants en clair.
- Problème 2 : toute requête GET vide sur l'API retourne désormais une erreur 400 Bad Request, empêchant tout accès direct sans paramètre.